NapCart ERD v1

Entity Relationship Design

Version: v1.0 Date: 2026-05-28 Status: Draft Author: Codex with Naptime AI

1. Purpose

This document defines the first database design for NapCart based on the finalized PRD and MVP decisions.

The schema is designed to be:

- multi-tenant-ready
- branch-aware
- WhatsApp-routing-aware
- guest-checkout-friendly
- future-ready without unnecessary MVP complexity

2. Design Principles

- Every operational record belongs to a restaurant
- Every order belongs to one branch
- Restaurant staff remains WhatsApp-first
- The dashboard/database is the system of record
- Orders must retain snapshot data even if menu items later change
- Customer identity uses internal IDs, with normalized phone number as the main matching key
- Branch-level configuration is required for routing and future scale

3. Schema Scope

3.1 Included in ERD v1

- Restaurants
- Branches
- Admin users

- Restaurant settings
- WhatsApp connections
- Categories
- Products
- Product variants
- Add-on groups
- Add-ons
- Delivery zones
- Customers
- Customer addresses
- Orders
- Order items
- Order item add-ons
- Order status history
- WhatsApp message logs

3.2 Intentionally Deferred

- Rider tables
- Coupon tables
- Loyalty tables
- Tax configuration tables
- Scheduled order tables
- Customer accounts/auth tables
- Full media asset library

4. High-Level Domain Map

```
Tenant Domain
  Restaurant
    -> Branch
    -> Admin User
    -> Restaurant Settings
    -> WhatsApp Connection

Catalog Domain
  Category
    -> Product
```

```
      -> Product Variant
      -> Add-on Group
        -> Add-on

Operations Domain
  Customer
    -> Customer Address
  Order
    -> Order Item
      -> Order Item Add-on
    -> Order Status History
    -> WhatsApp Message Log

Delivery Domain
  Branch
    -> Delivery Zone
```

5. Relationship Visualization

5.1 Core Relationship Diagram

```
Restaurant
  |--< Branch
  |--< AdminUser
  |--1 RestaurantSettings
  |--< Category
  |--< Product
  |--< Customer
  |--< Order
  |--< WhatsAppConnection

Branch
  |--< DeliveryZone
  |--< Order
  |--0..1 WhatsAppConnection

Category
  |--< Product

Product
  |--< ProductVariant
  |--< AddonGroup
  |--< OrderItem

AddonGroup
```

```
  |--< Addon

Customer
  |--< CustomerAddress
  |--< Order

Order
  |--< OrderItem
  |--< OrderStatusHistory
  |--< WhatsAppMessageLog

OrderItem
  |--< OrderItemAddon
```

5.2 WhatsApp Routing Model

```
Restaurant
  -> Branch
      -> WhatsApp Connection

Order placed
  -> branch_id selected
  -> branch WhatsApp connection loaded
  -> order message sent to that destination

If one shared number exists:
  multiple branches may still map to the same WhatsApp
connection
  while branch labels remain visible in the message payload
```

6. Key Enumerations

6.1 Fulfillment Type

- `delivery`
- `pickup`

6.2 Order Status

- `pending_confirmation`
- `confirmed`
- `cancelled`

6.3 Payment Method

- `cash_on_delivery`
- `cash_on_pickup`

6.4 Payment Status

- `unpaid`
- `paid`

MVP note:

- payment status exists mainly as a future-ready field
- operationally, MVP uses cash flows only

6.5 WhatsApp Message Direction

- `outbound`
- `inbound`

6.6 WhatsApp Message Status

- `queued`
- `sent`
- `delivered`
- `failed`
- `received`
- `processed`

7. Table Specifications

7.1 restaurants

Purpose:

- top-level tenant record

Fields:

```
id (uuid, pk)
name
slug
logo_url
support_phone
```

```
default_currency
default_language
timezone
contact_email nullable
is_active
created_at
updated_at
```

Notes:

- `slug` supports tenant-aware routing later
- `default_language` remains `English` in MVP, but keeps the schema future-ready

7.2 branches

Purpose:

- operational branch used for customer selection and order routing

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
name
slug
phone nullable
address_text
latitude nullable
longitude nullable
is_active
is_accepting_orders
is_temporarily_closed
display_order
created_at
updated_at
```

Notes:

- branch selection is manual in MVP
- branch availability is operationally important

7.3 branch_operating_hours

Purpose:

- define opening hours without bloating the branch table

Fields:

```
id (uuid, pk)
branch_id (fk -> branches.id)
day_of_week
open_time nullable
close_time nullable
is_closed
created_at
updated_at
```

Notes:

- supports "closed now" + structured weekly hours
- `is_closed = true` allows full-day closure

7.4 restaurant_settings

Purpose:

- store tenant-level operational settings

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id, unique)
is_accepting_orders
is_globally_closed
minimum_order_amount nullable
pickup_enabled
delivery_enabled
show_branch_selection
customer_notifications_enabled
tax_enabled
created_at
updated_at
```

MVP note:

- `tax_enabled` can default false until the tax module exists

7.5 admin_users

Purpose:

- dashboard login access

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
name
email
password_hash or auth_provider_id
is_active
created_at
updated_at
```

Notes:

- MVP supports one main admin account per restaurant
- role-based permissions are deferred

7.6 whatsapp_connections

Purpose:

- per-restaurant or per-branch WhatsApp destination configuration

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
branch_id nullable (fk -> branches.id)
provider
business_name
whatsapp_business_account_id nullable
phone_number_id nullable
display_phone_number
api_base_url nullable
access_token_encrypted nullable
webhook_verify_token_encrypted nullable
is_active
is_default_for_restaurant
```

```
created_at
updated_at
```

Notes:

- branch-specific routing uses `branch_id`
- shared routing uses a restaurant-level or repeated mapped connection
- mock mode can exist without real credentials

7.7 categories

Purpose:

- menu grouping

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
name
slug
description nullable
sort_order
is_active
created_at
updated_at
```

7.8 products

Purpose:

- menu item master record

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
category_id (fk -> categories.id)
name
slug
description nullable
image_url nullable
base_price
is_active
```

```
is_available
delivery_available
pickup_available
display_order
created_at
updated_at
```

Notes:

- `is_active` is structural visibility
- `is_available` supports temporary unavailability

7.9 product_variants

Purpose:

- variations like small/medium/large

Fields:

```
id (uuid, pk)
product_id (fk -> products.id)
name
sku nullable
price_delta nullable
fixed_price nullable
is_default
is_active
sort_order
created_at
updated_at
```

Rules:

- either `price_delta` or `fixed_price` is used
- default variant is optional if base product already covers the default case

7.10 addon_groups

Purpose:

- group related add-ons per product

Fields:

```
id (uuid, pk)
product_id (fk -> products.id)
name
min_select
max_select
is_required
sort_order
is_active
created_at
updated_at
```

7.11 addons

Purpose:

- individual selectable add-ons

Fields:

```
id (uuid, pk)
addon_group_id (fk -> addon_groups.id)
name
price
is_active
sort_order
created_at
updated_at
```

7.12 delivery_zones

Purpose:

- branch-level delivery fee and coverage rule

Fields:

```
id (uuid, pk)
branch_id (fk -> branches.id)
name
max_distance_km nullable
area_label nullable
fee
minimum_order_amount nullable
```

```
is_active
sort_order
created_at
updated_at
```

Notes:

- supports simple zone logic such as 5 km, 8 km, or named local areas
- future geospatial logic can build on top of this table

7.13 customers

Purpose:

- internal customer record

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
name
normalized_phone
raw_phone_input
email nullable
total_orders_count
first_order_at nullable
last_order_at nullable
notes nullable
created_at
updated_at
```

Rules:

- unique index on `(restaurant_id, normalized_phone)`
- customer name is not a unique key

7.14 customer_addresses

Purpose:

- future-friendly customer address storage

Fields:

```
id (uuid, pk)
customer_id (fk -> customers.id)
label nullable
address_text
latitude nullable
longitude nullable
delivery_notes nullable
is_default
created_at
updated_at
```

MVP note:

- checkout uses free-text entry
- table supports future repeat-address behavior later

7.15 orders

Purpose:

- main order record

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
branch_id (fk -> branches.id)
customer_id nullable (fk -> customers.id)
whatsapp_connection_id nullable (fk -> whatsapp_connections.id)
order_number
status
fulfillment_type
payment_method
payment_status
customer_name_snapshot
customer_phone_snapshot
address_text_snapshot nullable
latitude_snapshot nullable
longitude_snapshot nullable
delivery_notes nullable
branch_name_snapshot
subtotal
delivery_fee
discount_total
tax_total
```

```
grand_total
currency
placed_at
confirmed_at nullable
cancelled_at nullable
created_at
updated_at
```

MVP defaults:

- `discount_total = 0`
- `tax_total = 0`
- `payment_status = unpaid`

Key rules:

- order starts as `pending_confirmation`
- it becomes `confirmed` or `cancelled` only after staff action

7.16 order_items

Purpose:

- ordered product lines with full snapshots

Fields:

```
id (uuid, pk)
order_id (fk -> orders.id)
product_id nullable (fk -> products.id)
product_name_snapshot
variant_name_snapshot nullable
unit_price
quantity
line_total
item_notes nullable
created_at
updated_at
```

Notes:

- `product_id` may be nullable long-term if product is ever removed
- snapshots preserve historical integrity

7.17 order_item_addons

Purpose:

- selected add-ons attached to an order item

Fields:

```
id (uuid, pk)
order_item_id (fk -> order_items.id)
addon_name_snapshot
addon_price_snapshot
quantity
line_total
created_at
updated_at
```

7.18 order_status_history

Purpose:

- audit trail of status transitions

Fields:

```
id (uuid, pk)
order_id (fk -> orders.id)
old_status nullable
new_status
change_source
changed_by_admin_user_id nullable (fk -> admin_users.id)
changed_at
notes nullable
```

Change source examples:

- `system`
- `whatsapp_staff_action`
- `admin_dashboard`

7.19 whatsapp_message_logs

Purpose:

- full message audit and routing visibility

Fields:

```
id (uuid, pk)
restaurant_id (fk -> restaurants.id)
branch_id nullable (fk -> branches.id)
order_id nullable (fk -> orders.id)
whatsapp_connection_id nullable (fk -> whatsapp_connections.id)
direction
message_type
provider_message_id nullable
template_name nullable
payload_json
response_json nullable
status
error_message nullable
sent_at nullable
received_at nullable
processed_at nullable
created_at
updated_at
```

Notes:

- critical for debugging, retries, and auditability

8. Index and Constraint Recommendations

8.1 Unique Constraints

```
restaurants.slug
restaurant_settings.restaurant_id
customers (restaurant_id, normalized_phone)
orders (restaurant_id, order_number)
whatsapp_connections (restaurant_id, branch_id,
display_phone_number) as needed by exact implementation
```

8.2 High-Value Indexes

```
branches.restaurant_id
products.restaurant_id
products.category_id
delivery_zones.branch_id
```

```
orders.restaurant_id
orders.branch_id
orders.customer_id
orders.status
orders.placed_at
order_items.order_id
whatsapp_message_logs.order_id
whatsapp_message_logs.provider_message_id
```

9. MVP vs Future-Ready Design Notes

9.1 Active in MVP

- restaurants
- branches
- operating hours
- settings
- admin user
- menu tables
- delivery zones
- customers
- orders
- order items
- status history
- WhatsApp logs

9.2 Future-Ready Fields Included Now

- latitude/longitude snapshots
- tax_total
- discount_total
- payment_status
- branch/restaurant-level WhatsApp mapping
- customer address storage beyond a single one-off order

9.3 Intentionally Not Added Yet

- coupon tables
- rider tables
- loyalty program tables

- customer auth tables
- tax config tables

Reason:

- these would add structural complexity without current MVP value

10. ERP-Level Logic Notes

These are not tables, but rules the implementation must respect:

- Product availability check happens before order creation
- Restaurant and branch availability check happens before order creation
- Delivery zone and minimum order rules must be validated server-side
- Order snapshots must be written at transaction time
- WhatsApp send attempt must happen only after order persistence succeeds
- Staff Confirm / Cancel action must update both `orders.status` and `order_status_history`

11. Example Order Flow Mapped to Tables

```
1. Customer chooses branch
   -> branches

2. Customer browses menu
   -> categories, products, variants, add-ons

3. Customer submits order
   -> customers (find or create)
   -> customer_addresses (optional create/update)
   -> orders
   -> order_items
   -> order_item_addons
   -> order_status_history

4. System sends WhatsApp message
   -> whatsapp_connections
   -> whatsapp_message_logs

5. Staff confirms or cancels
   -> orders.status update
```

```
    -> order_status_history insert
    -> whatsapp_message_logs inbound/processed record
```

12. Open Design Notes for Next Artifact

The ERD is now ready enough to support the next document:

- API specification
- architecture specification
- implementation sequencing

Remaining design work belongs in the architecture phase, not the ERD phase:

- exact webhook action payload format
- exact branch-routing resolution logic
- exact auth implementation choice
- exact storage provider choice for images

13. Final Recommendation

NapCart ERD v1 should be implemented as a normalized relational schema with:

- strong tenant isolation
- branch-aware routing
- robust snapshotting for orders
- minimal but future-ready operational fields
- explicit WhatsApp audit logging

This is the right level of complexity for MVP because it supports:

- one real restaurant launch
- future onboarding of new restaurant clients
- future branch expansion
- future integration with existing websites